

CITS3003 Graphics & Animation

Project Report

Project Overview

The project implements a scene editor allowing users to be able to create, manipulate and customise 3D objects and scenes. The editor supports various features such as camera control, object transformation, lighting systems, material editing, and shader-based rendering improvements.

Content

Task A -----	pg 2
Task B -----	pg 3
Task C -----	pg 4
Task D -----	pg 5
Task E -----	pg 6
Task F -----	pg 7
Bonus Challenge -----	
Task G -----	pg 9
Task H -----	pg 10
Task I -----	pg 12

Required Tasks

Task A: Panning camera

The goal of this task was to complete the `PanningCamera` so that the camera can be controlled interactively. The required controls include right mouse dragging for rotation, middle mouse dragging for panning, scroll wheel zooming, and resetting the camera using the `R` key.

The main implementation change was replacing the old translation-only view matrix with a camera position calculated from `pitch`, `yaw`, `distance`, and `focus_point`.

direction vector → camera position → view matrix



Test:

Distance: 8; ✓

Pitch: -45; ✓

Yaw: 315; ✓

Near Plane: 0.01; ✓

FOV: 90; ✓

Gamma: 2.20; ✓

Which are all the same as what is in the video.

Reset working ✓

Right mouse button ✓

Left mouse button ✓

R key ✓

mouse scroll ✓

Task B: Entity rotation

The goal of this task was to make the `euler_rotation` field in `LocalTransformComponent` actually affect how scene elements are rendered. Before this change, the rotation values in the UI didn't really rotate the object. That's because the model matrix only included translation and scaling but not rotation. Although `euler_rotation` was stored and updated correctly through the ImGui rotation controls, it was never used when calculating the final model matrix. Therefore, the rendered object did not rotate.

```
glm::mat4 EditorScene::LocalTransformComponent::calc_model_matrix() const {
    glm::mat4 model = glm::mat4(1.0f);

    model = glm::translate(model, position);

    model = glm::rotate(model, euler_rotation.x, glm::vec3(1.0f, 0.0f, 0.0f));
    model = glm::rotate(model, euler_rotation.y, glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::rotate(model, euler_rotation.z, glm::vec3(0.0f, 0.0f, 1.0f));

    model = glm::scale(model, scale);

    return model;
}
```

Problem: “do something sensible” for the Euler rotation problem, we choose to use XYZ rotation order.

Ideas: A possible improvement would be to track the order in which the user edits the rotation values, and then compose the rotation matrices using that same order? With its own pros and cons: the same X/Y/Z rotation values could produce different results depending on the order in which the user entered them. And it may not be intuitive for users.

Task C: Clipping Plane

The original code caused the objects to be incorrectly cut off when the camera was moved close to them, preventing close up views of the scene objects. This was caused by the camera projection setup not fully using the near plane values when building the final view/projection behaviour. This meant the visible camera volume did not match the user's selected near-plane setting.

Thus, the update function of the `PanningCamera` and `FlyingCamera` camera file logics were changed so that the near and far clipping plane values, shown in the UI, are used when building the projection/view setup. In each camera's update path, the projection matrix is now built using:

```
glm::infinitePerspective(fov,  
window.get_framebuffer_aspect_ratio(), near)
```

This means the `near` value comes from the camera's UI-controlled near-plane setting instead of an incorrect hard-coded value. Therefore, when the user changes the near plane in the UI, the camera projection now uses that value directly.

UI near plane value → `glm::infinitePerspective(...)` → camera projection matrix → correct clipping behaviour

This fixed the close-up clipping issue because the projection now respects the actual near clip plane. Scene objects are no longer incorrectly cut off when the camera moves near them, giving better control when inspecting models, lights, and scene details up close.

Tests:

Close-up camera view works without objects being cut off too early ✓

Near plane value in the UI affects the camera projection ✓

Both `PanningCamera` and `FlyingCamera` respect the near-plane setting ✓

Camera clipping behaviour is more consistent ✓

Task D: Material Properties

The original code didn't fully expose the materials values for ambient, diffuse, emissive and specular tints as well as shininess. This prohibited the users from controlling how objects responded to light in the scene from the editor.

This was implemented by updating the `SceneElement.cpp` to replace the placeholder material UI with real ImGui controls, including:

- Diffuse Tint
- Diffuse Intensity
- Specular Tint
- Specular Intensity
- Ambient Tint
- Ambient Intensity
- Shininess
- Emission Tint
- Emission Intensity

ImGui material controls → material property values → `update_instance_data()`
→ real-time rendered result

The scene editor can now change material colours and strengths interactively. When a material edit is detected, `update_instance_data()` applies the new values so the object's appearance updates in real time.

The ambient change affects how visible an object is when it is in a less lit area. The diffuse colour affects the base colour of the object under lighting and shininess affects the size and strength of specular highlights, making the objects seem more or less reflective. For emissive objects, the color is treated differently as it represents light from objects rather than light reflected from another source. These three components come together to form the final illustration.

Tests:

Changing diffuse tint changes object colour ✓

Changing specular/shininess changes highlight appearance ✓

Changing emission tint/intensity affects emissive appearance ✓

Material controls update interactively in the editor ✓

Task E: Texture scaling

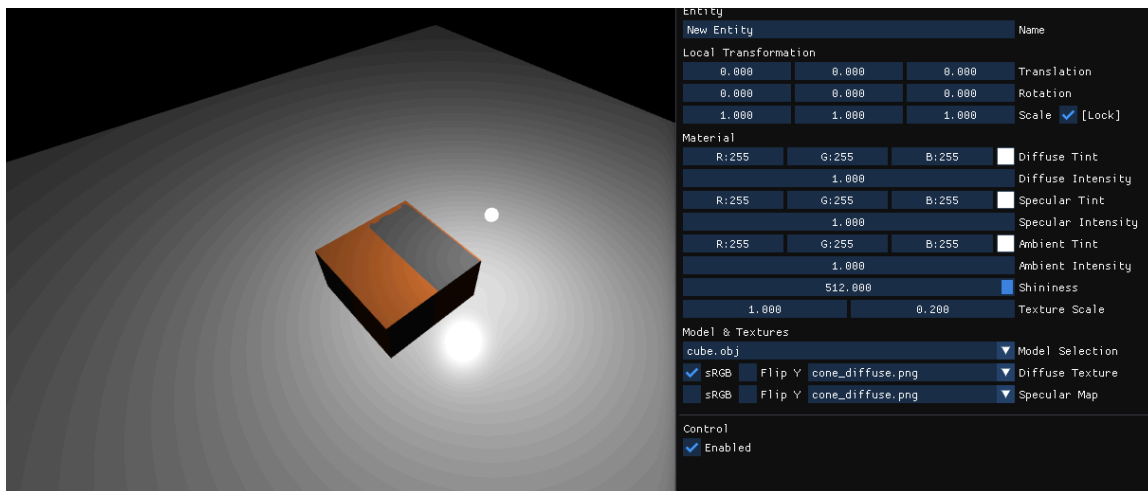
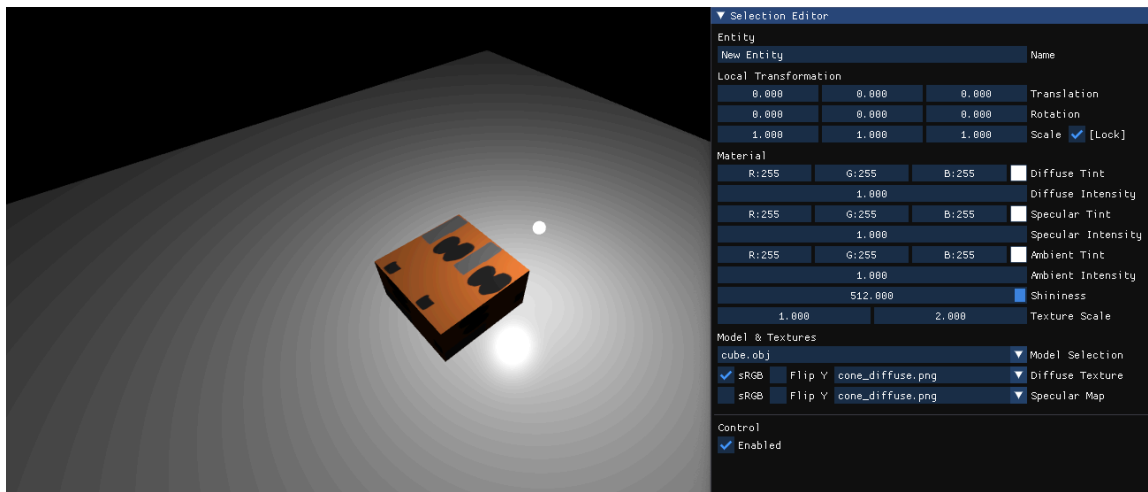
The goal of this task was to add texture scaling for both `EntityElement` and `AnimatedEntityElement`, allowing textures to display correctly on model surfaces. Before this, texture could not scale or repeat.

We use uniform `vec2 texture_scale;` for scaling.

```
vertex_out.texture_coordinate = texture_coordinate *  
texture_scale;
```

allows UV coordinates to be scaled inside the vertex shader.

A new ImGui control was added: `ImGui::DragFloat2("Texture Scale", ...)` at `src/scene/editor_scene/SceneElement.cpp`



Test: PASS. Both bigger/smaller scale values work for both Entity and Animated entity.

Task F: Light Attenuation

The original code's point light affected far-away surfaces too strongly, thus needed to modify the point light calculation in `res/shaders/common/lights.glsl` so that light intensity decreases with distance. Also, without attenuation, nearby and far-away fragments could receive too much similar light contribution. Hence, the shader logic was changed to calculate distance between the point light and fragment positions and then applied to reduce the light contribution.

This was implemented by updating `res/shaders/common/lights.glsl` so the point light calculation computes the distance between the point light and the fragment position. This distance is then used in an attenuation formula:

distance → attenuation factor → reduced diffuse/specular light

The attenuation formula used was:

```
attenuation = 1 / (1 + 0.09 * d + 0.032 * d2)
```

The attenuation value is then applied to the diffuse and specular components of the point light. As a result, fragments close to the light appear brighter, while fragments further away receive less light.

Tests:

Objects close to point light are brighter ✓

Objects far from point light are dimmer ✓

Diffuse and specular components fade with distance ✓

Lighting looks more realistic than before ✓

Bonus Challenge:

In the normal implementation, the point light attenuation was added so that the light intensity decreases based on distance but the attenuation was still changing by formula.

This bonus implementation improved the lighting control by allowing the attenuation parameter to be changed through the editor UI. Instead of using one hard-coded attenuation value for every point light, the value is stored as part of the point light properties and then sent to the shader as a uniform, meaning each point light can have its own falloff behaviour depending on how the user adjusts it.

The main implementation change was adding a new `glm::vec3 attenuation` field to `PointLight` in `Lights.h`. This vector stores the three attenuation terms:
constant attenuation → linear attenuation → quadratic attenuation

The default values were set to match the previous hard-coded formula, so the original attenuation behaviour was preserved unless the user changed the values. The shader was then updated in `lights.glsl` so that the attenuation calculation uses each light's own attenuation values instead of fixed constants. The attenuation calculation is now based on the selected point light's stored parameters:

point light attenuation values → shader attenuation formula → custom light falloff

The C++ shader setup in `BaseLitEntityShader.cpp` was also updated so the attenuation data is passed from the application to the GPU. This connects to the course concept of using uniforms to send application-side data into GLSL shaders. The light properties are stored and edited on the C++ side, then used by the GLSL shader during the lighting calculation.

UI controls were also added in `PointLightElement.cpp` and `OrbitingPointLightElement.cpp` (task 1 addition) allowing the user to edit the constant, linear, and quadratic attenuation values directly in the editor. JSON serialization was also added, so these attenuation settings can be saved and loaded with the scene.

UI attenuation sliders → PointLight attenuation property → shader uniform → editable light falloff → saved/loaded scene setting

With this bonus implementation, each point light can now have independent attenuation behaviour. Increasing the linear or quadratic attenuation values makes the light fade faster and affect a smaller area. Decreasing them allows the light to reach further and illuminate a wider area. This gives the editor finer control over lighting style and makes it possible to create different types of point lights in the same scene.

Tests:

- Attenuation can be changed from the UI ✓
- Constant, linear, and quadratic sliders work ✓
- Increasing attenuation makes the light fade faster ✓
- Decreasing attenuation makes the light reach further ✓
- Different point lights can have different falloff behaviour ✓
- Attenuation settings can be saved and loaded through JSON ✓

Task G: Shading

In the original code, the light calculation where based in the vertex shader, so directions were calculated for each vertex (per-vertex lighting calculates light at vertices and then interpolates the result across the triangle). This caused less accurate/realistic lighting, especially on large surfaces.

This was changed by updating the lit entity shaders so that the vertex shader outputs world-space fragment position and normal, while the fragment shader performs the actual lighting calculation.

world-space position + normal → fragment shader → per-fragment lighting

In the shader changes, the old per-vertex `LightingResult` output was removed. Instead, the vertex shader passes the required world-space data to the fragment shader. The fragment shader then reconstructs the lighting calculation data, including view direction, material properties, and point light data, and calls the total lighting calculation per fragment.

By calculating lighting per fragment, the light response becomes more detailed and visually accurate.

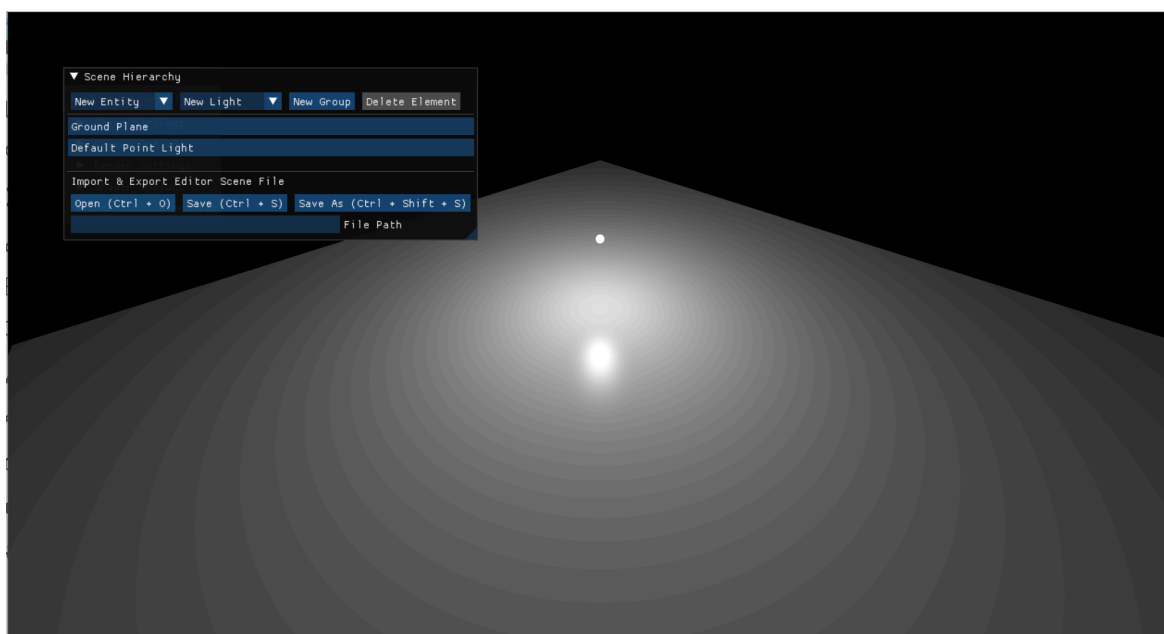
Tests:

Lighting calculation happens in fragment shader ✓

Ground/light interaction looks more accurate ✓

Point light shine is more detailed ✓

Large surfaces no longer rely only on interpolated vertex lighting ✓



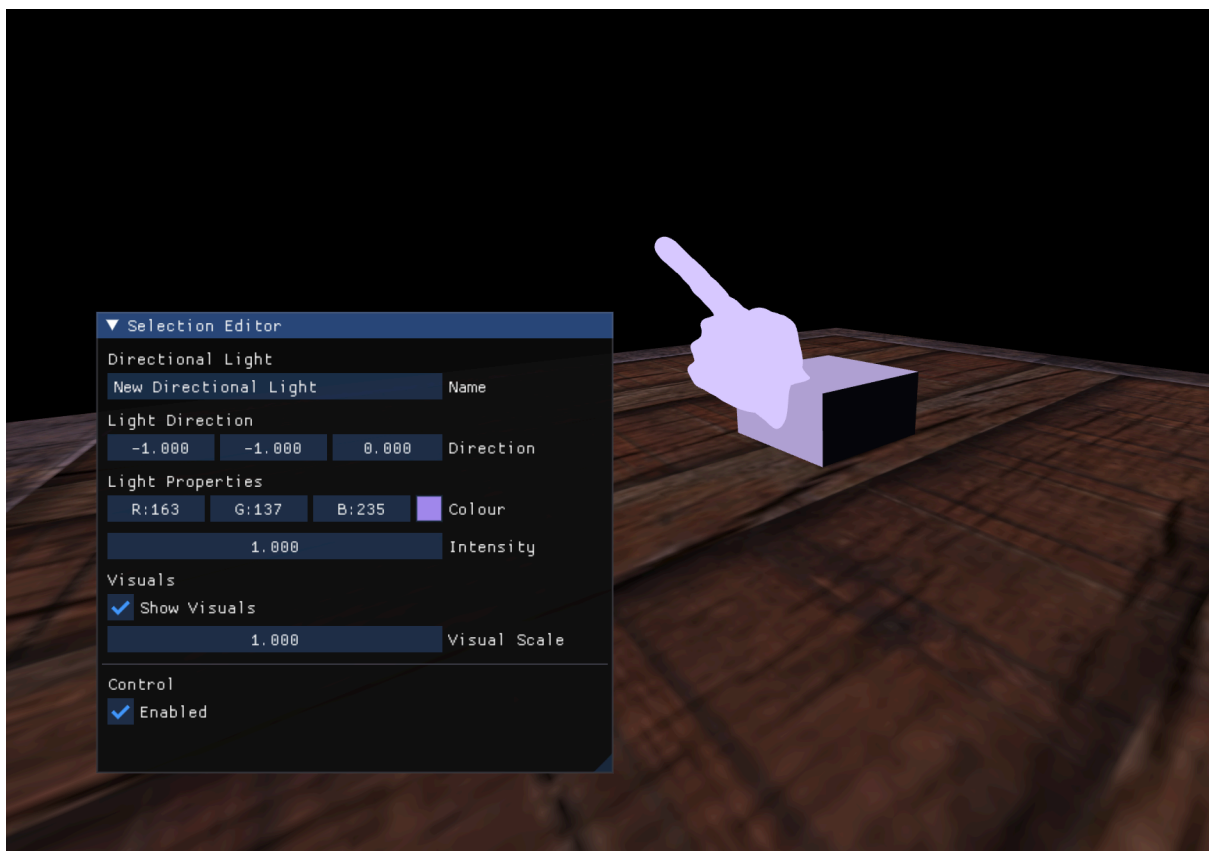
Task H: Light source

The goal of this task was to add a second light type(directional light) to the renderer.

I implemented the directional light by following the existing point light structure. A new `DirectionalLight` type was added to the C++ light system, and the render scene was extended so it can store and remove multiple directional lights. I also created a new `DirectionalLightElement` for the editor scene, which allows the user to create a directional light, edit its direction, colour, intensity, and visual marker from the UI.

The renderer and shader pipeline were also updated. I added support for `DirectionalLightData` in `lights.glsl`, added a directional light calculation function, and updated both `entity` and `animated_entity` fragment shaders so normal entities and animated entities can both receive directional lighting. `EntityRenderer`, `AnimatedEntityRenderer`, and `BaseLitEntityShader` were updated to pass directional light data to the GPU using a uniform buffer. The editor registration was also updated so directional lights can be created from the UI and correctly saved and loaded from scene files.

IMPROVEMENT: visual indicator 🧐👉



Replacing the original sphere marker with a custom `pointer.obj` mode (online resource) I. Unlike the sphere, the pointer more clearly communicates the actual light direction in the scene, making it easier to identify and debug the light direction in the editor.

Reference for the pointer.obj : <https://free3d.com/3d-model/pointer-v1--510788.html>

Testing: Object brightness changes with the light direction and surface angle instead of the distance to the light source, confirming that it behaves as a directional light rather than a point light.

Task I: New Feature - Dynamic Orbiting Day-Night Cycle Light and Atmospheric Change

In the original scene, the lighting and atmosphere were mostly static, so the scene did not change much over time unless the user manually edited values. To make the scene feel more dynamic and visually expressive, a new feature was implemented where the light orbits around the scene and changes colour over time to create a day-night cycle effect as well as an atmospheric fog change so the scene's appearance changes depending on the lighting state.

This implementation improved the scene by using position-based updates to modify the light's position and colour during rendering. Instead of the light staying fixed in one position, its position is updated each frame so that it moves around the scene in an orbit.

The orbiting point light was implemented based on point light (aka borrowing shaders from it).

Instead of adding a new shader light type, the C++ code dynamically updates the point light position over time. The `current_position()` function calculates circular movement using `cos` and `sin`, where `orbit_radius` controls the size of the orbit, `orbit_duration` controls the rotation speed, and `orbit_tilt_angle` controls the tilt of the orbit path. The calculated position is then assigned to `light->position` in `update_instance_data()`.

```
light->position = glm::vec3(transform[3]);
if (visible) {
    light_sphere->instance_data.model_matrix = transform * glm::scale(glm::vec3{0.1f * visual_scale});
} else {
    // Throw off to infinity as a hacky way to make model invisible
    light_sphere->instance_data.model_matrix = glm::scale(glm::vec3{std::numeric_limits<float>::infinity()}) *
}

glm::vec3 colour = glm::vec3(light->colour);
```

We also want to extend the existing Orbiting Point Light system by allowing the light colour to change dynamically while moving along its orbit.

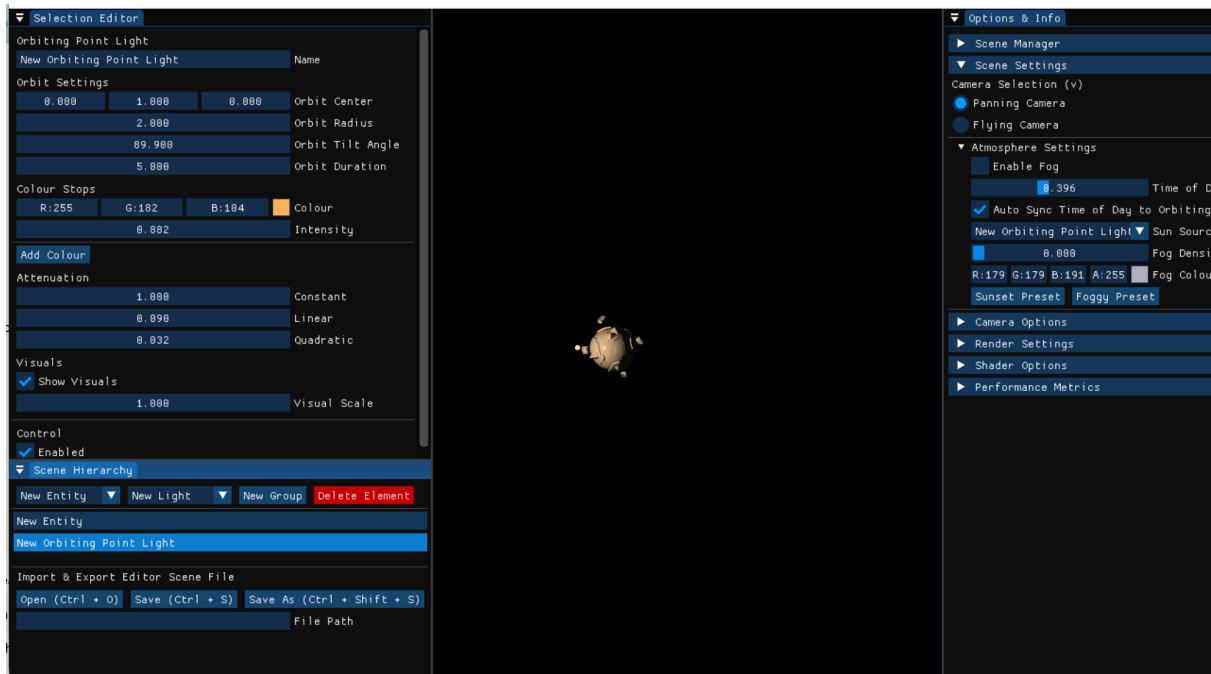
```
float safe_duration = std::max(orbit_duration, 0.001f);
float progress = std::fmod(time_seconds / safe_duration, 1.0f);
if (progress < 0.0f) {
    progress += 1.0f;
}

float scaled_progress = progress * static_cast<float>(orbit_colours.size());
int first_index = static_cast<int>(std::floor(scaled_progress)) % static_cast<int>(orbit_colours.size());
int second_index = (first_index + 1) % static_cast<int>(orbit_colours.size());
float blend = scaled_progress - std::floor(scaled_progress);

return glm::mix(orbit_colours[first_index], orbit_colours[second_index], blend);
```

We also implement colour Interpolation to smoothly blend between neighbouring colours. We updated the light colour every frame, which allows us to use the same shader as point light does later.

IDEA: Allowing users to manually set time/position for every colour would significantly increase the complexity for both UI and algorithm. We solved this by evenly distributing all colour stops around the orbit. To compensate for the lack of manual time/position control, users can duplicate colours to extend their duration.



The **day-night colour-changing** part is performed as the final colour of a surface depends on the light source and material interaction, changing the light colour affects how objects are shaded. For example, warmer colours can create a sunrise or sunset effect, while darker or cooler colours can create a night-time effect. The lighting calculation is different light properties over time.

This was implemented by updating `EditorScene.cpp` so that only `OrbitingPointLightElement` receives the new time-of-day light colour. Instead of directly setting every light's colour, the code now updates:

```
Orbiting_light -> orbit_colours = { light_colour }
```

This means the colour-changing behaviour is attached specifically to orbiting lights rather than affecting all scene elements.

time-of-day colour → `OrbitingPointLightElement` → `orbit_colours` → dynamic scene lighting

The **atmospheric fog** part was also implemented in `EditorScene.cpp` under the Atmosphere Settings UI. Three controls were added: `Enable Fog`, `Fog Density`, and `Fog Colour`. `Enable Fog` allows the user to turn the fog effect on or off, `Fog Density` controls how strong the fog effect is, and `Fog Colour` allows the user to

choose the RGBA colour used for the fog. When fog is enabled, the time-of-day light and ambient colours are blended toward the selected fog colour, using the fog density as the blend factor.

fog colour + fog density → blended light/ambient colour → atmospheric scene effect

This makes the scene more visually atmospheric as the fog colour affects the overall lighting mood. A low fog density keeps more of the original day-night lighting, while a higher density makes the scene appear more hazy.

The fog also relates to fragment shader logic as it affects the final visual colour of the scene. The normal shaded colour is influenced by the fog colour depending on the atmosphere settings, making far objects appear more faded and helping create depth in the scene. Combined with the orbiting light, the fog makes the environment feel more realistic because both the light source and the atmosphere change together.

The **auto-sync feature** was added to connect the time-of-day system with the orbiting light's movement. Instead of manually adjusting the time-of-day slider to match the orbiting light position, the user can enable `Auto Sync Time of Day to Orbiting Light`. When enabled, the system calculates the selected orbiting light's current world position, compares its local Y offset from the orbit centre, normalises it using the orbit radius, and maps that height to the `time_of_day` value.

orbiting light position → local Y offset → normalised height → `time_of_day`

Thus, the top of the orbit corresponds to daytime, the side of the orbit corresponds to sunrise or sunset, and the bottom of the orbit corresponds to night.

Hence, the `time-of-day` lighting automatically follows the orbiting light's vertical position, keeping the light movement, colour change, and atmospheric fog synchronised without requiring manual slider adjustment.

Overall, Task I extends the lighting system by combining animation, shading, and fragment colour blending. The orbiting light creates a moving day-night cycle, the colour changes affect how surfaces receive light, and the fog changes modify the final fragment colour to create atmospheric depth. This makes the scene more dynamic than a static lighting setup and demonstrates how shader-based rendering can be used creatively to control both lighting and atmosphere.